Lab 3 — Pthreads Mutex Locks and Semaphores in C on Linux

Operating Systems (SFWRENG/COMP SCI 3SH3), Fall 2026

Prof. Neerja Mhaskar

Lab Format

1. You must show your working solutions for Parts I and II of this lab to your TA for a grade.
2. The TA will review your solutions, ask questions about your work, and then enter your grade and feedback on Avenue.
3. If you are unable to present your work to your TA during your scheduled lab in the grading week, please contact both the instructor and your Lab TA to discuss possible accommodations either before or, at the latest, during the grading week. Requests for accommodations received after the class has moved on to a different lab will not be considered.
4. If you do not show your work to your Lab TA, you will get a zero (unless you provide an MSAF, in which case you will get a 5-day extension to get this lab graded).
5. It is your responsibility to follow up with your Lab TA to receive a grade and to ensure that your grade has been posted on Avenue.

Part I — Banking System Problem

Consider a simple banking system that maintains bank accounts for its users. Every bank account has a balance (represented by an integer variable `amount`). The bank allows deposits and withdrawals from these bank accounts, represented by the two functions `deposit` and `withdraw`. These functions are passed an integer value that is to be deposited or withdrawn from the bank account. Assume that a husband and wife share a bank account. The husband only withdraws from the account and the wife only deposits into the account using the `withdraw` and `deposit` functions respectively. A race condition is possible when the shared data (`amount`) is accessed by these two functions concurrently.

In this lab you are to write a C program that provides a critical-section solution to the Banking System Problem using mutex locks provided by the POSIX Pthreads API. In particular, your solution needs to do the following:

1. Take **two command line arguments**. The first argument is the amount to be deposited (an integer value) and the second argument is the amount to be withdrawn (an integer value).
2. Create a total of 6 threads that run concurrently using the Pthreads API.
   a. 3 of the 6 threads call the `deposit()` function, and
   b. 3 of the 6 threads call the `withdraw()` function.
3. Create the threads calling the `deposit()` function using the `pthread_create()` function. While creating these threads you need to pass the thread identifier, the attributes for the thread, the `deposit()` function, and the first integer command line argument `argv[1]` (which is the amount to be deposited).

4. Similarly, create the threads calling the `withdraw()` function.
5. To achieve mutual exclusion, use mutex locks provided by the Pthreads API.
6. You are to provide print statements that output an error message if an error occurs while creating threads, mutex locks, etc.
7. Your program should print the value of the shared variable `amount` whenever it is modified.
8. Finally, the parent thread should output the final `amount` value after all threads finish their execution.

Important

Make sure you use `pthread_join()` for all the threads created. This will ensure that the parent thread waits for all the threads to finish, and the final `amount` reported by `main` is correct for every execution of the program.

Notes:

1. See accompanying slides (Mutex and Semaphores in C.pdf) with this lab on the use of mutex locks provided by the Pthreads API and to use semaphores provided by the POSIX SEM extension.
2. You may see that the `amount` is negative. This could happen if the threads calling the `withdraw` function are scheduled to run on the CPU before the `deposit` function. This is acceptable for Part I of this lab.

Sample Output: `./PLmutex 100 50`

```
Withdrawal amount = -50
Withdrawal amount = -100
Withdrawal amount = -150
Deposit amount = -50
Deposit amount = 50
Deposit amount = 150
Final amount = 150
```

Part II

In this part you are to modify your C program created for **Part I** to ensure the following conditions are met:

1. Withdrawals don't take place if `amount` ≤ 0.
2. Deposits don't take place if `amount` ≥ 400.
3. The amount of money deposited or withdrawn at a given time is 100.
4. Your program should create a total of **10** threads that run concurrently. **7 of 10** threads call the `deposit()` function and **3 of 10** threads call the `withdraw()` function.

   In particular, your solution needs to do the following:

1. Take **one command line argument**. Since the amount of money withdrawn/deposited at a given time is 100, the value of the command line argument is 100.
2. You are to use mutex locks provided by the Pthreads API to achieve mutual exclusion as explained in Part I.
3. You are to use **two semaphores** to ensure **condition 1 and condition 2** are met.
4. Additionally, you need to set the initial value of these semaphores correctly for your solution to work.
5. You are to provide print statements that output an error message if an error occurs while creating threads, mutex locks, semaphores, etc.
6. You are to provide print statements in the `deposit` and `withdraw` functions that output the value of the shared variable `amount` after each modification.
7. You are to provide print statements at the beginning of the `deposit()` and `withdraw()` functions. This way you can see (through the print statements) when threads begin their execution in their functions.
8. Finally, the parent thread should output the final `amount` value after all threads finish their execution. Since the `deposit` function gets executed 7 times and the `withdraw()` function gets executed 3 times, the correct final amount $= 7 \times 100 - 3 \times 100 = 700 - 300 = 400$.

Important

Make sure you use `pthread_join()` for all the threads created. This will ensure that the parent thread waits for all the threads to finish, and the final `amount` reported by this thread is correct for every execution of the program.

Notes:

1. See accompanying slides (Mutex and Semaphores in C.pdf) with this lab on the use of mutex locks provided by the Pthreads API and to use semaphores provided by the POSIX SEM extension.
2. **If you see negative `amount` values, your solution is incorrect.**

Sample Output: `./PLsem 100`

```
Executing deposit function
Amount after deposit = 100
Executing Withdraw function
Amount after Withdrawal = 0
Executing Withdraw function
Executing Withdraw function
Executing deposit function
Amount after deposit = 100
Amount after Withdrawal = 0
Executing deposit function
Amount after deposit = 100
Amount after Withdrawal = 0
Executing deposit function
Amount after deposit = 100
Executing deposit function
```

```
Amount after deposit = 200
Executing deposit function
Amount after deposit = 300
Executing deposit function
Amount after deposit = 400
Final amount = 400
```

End of Lab 3